

Software Requirements Specification

for

FlightSound

Version 1.8 approved

Prepared by Agustin, Angel, Carter, Derrick, Isaac, Kassandra, Ruby, Daphne, and
Tiffany

Insert company / project sponsor name here...

May 8, 2026

Table of Contents

Table of Contents.....	pg 2
Revision History.....	pg 3
1. Introduction.....	pg 4
1.1. Purpose.....	pg 4
1.2. Intended Audience and Reading Suggestions.....	pg 4
1.3. Product Scope.....	pg 4
1.4. Definitions, Acronyms, and Abbreviations	pg 4
1.5. References.....	pg 5
2. Overall Description.....	pg 6
2.1. System Analysis.....	pg 6
2.2. Product Perspective.....	pg 6
2.3. Product Functions.....	pg 7
2.4. User Classes and Characteristics.....	pg 9
2.5. Operating Environment.....	p 10
2.6. Design and Implementation Constraints.....	p 11
2.7. User Documentation.....	p 11
2.8. Assumptions and Dependencies.....	p 12
2.9. Apportioning of Requirements.....	p 12
3. External Interface Requirements.....	p 13
3.1. User Interfaces.....	p 13
3.2. Hardware Interfaces.....	p 15
3.3. Software Interfaces.....	p 15
3.4. Communications Interfaces.....	p 16
4. Requirements Specification.....	p 17
4.1. Functional Requirements.....	p 17
4.2. External Interface Requirements.....	p 21
4.3. Logical Database Requirements.....	p 22
4.4. Design Constraints.....	p 22
5. Other Nonfunctional Requirements.....	p 24
5.1. Performance Requirements.....	p 24
5.2. Safety Requirements.....	p 24
5.3. Security Requirements.....	p 24
5.4. Software Quality Attributes.....	p 25
5.5. Business Rules.....	p 25
6. Legal and Ethical Considerations.....	p 27
Appendix A: Glossary.....	p 28
Appendix B: Analysis Models.....	p 29
Appendix C: To Be Determined List.....	p 31

Revision History

Revision #	Change Description	Revision Date	Revised By	Approved By
1.0	Initial Draft	2/15/2026	Whole Group	Whole Group
1.1	Draft	2/16/2026	Kassandra, Ruby, Agustin	Whole Group
1.2	Draft	2/16/2026	Derrick, Angel	Whole Group
1.3	Final Draft	2/17/2026	Whole Group	Whole Group
1.4	Updated SRD to sync with GitHub Features and Stories	3/13/2026	Tiffany	Augstin, Angel, Ruby
1.5	Updated SRD to reflect working prototype implementation including AirLabs API integration, Google Maps visualization, backend/frontend connection, and noise calculation features	3/26/2026	Tiffany	Whole Group
1.6	Updated SRD section 1 3,4, and 5 to accurately reflect the requirements and non-functional requirements that have been implemented successfully in the program.	4/29/2026	Ruby, Agustin	Whole Group
1.7	Updated Sections 2, 6, and Appendix A to align with current system implementation, clarify legal and ethical considerations, and add glossary definitions.	4/30/2026	Kassandra	Angel
1.8	Updated Appendix B to showcase the expected personas that will interact with the system, taking into consideration the requirements that have been implemented.	5/1/2026	Isaac	Whole Group

1. Introduction

1.1 Purpose

The purpose of this document is to describe the software requirements for FlightSound, a web-based application that allows users to view real-time aircraft activity and estimated noise levels on an interactive map. The system retrieves live flight data using the AirLabs API and displays aircraft positions using the Google Maps JavaScript API.

The current version reflects a working prototype where users can search for an airport, view nearby flights, see estimated noise levels based on flight data, create an account through Google OAuth, and be able to save their location. This document defines what the system currently does and outlines additional planned features for future development.

1.2 Intended Audience and Reading Suggestions

This document is mainly for the developers working on FlightSound, but it can also be related to other people involved in the project. Here is a breakdown of who might read this and what they should focus on:

Developers should read the entire document since they need to understand the requirements before building the website.

Testers should focus on section 3 because that section describes how the website interacts with users and outside systems.

Project managers reviewing the project should focus on sections 1 and 2 to get the overall idea of FlightSound and what the goal of this product is.

Users should focus on sections 1 and 2 to understand the purpose of Flightsound and how it can benefit them.

We recommend reading the document from start to finish since each section builds on the one before it, and together all of the sections give a complete overview of the project's goals, description and interface requirements.

1.3 Product Scope

FlightSound is a web-based application that displays real-time flight activity and estimated aircraft noise levels on an interactive map. Users can search for an airport and view nearby aircraft along with their noise impact.

In the current prototype, the system successfully retrieves live flight data from the AirLabs API, processes the data in the backend, and displays aircraft markers on a Google Maps interface. Noise levels are calculated based on the user's actual position in relation to airplanes. Weather integration has also been completed with the weather being displayed in its own section next to the map. Users are also able to create an account which will save their location and selected airport.

1.4 Definitions, Acronyms, and Abbreviations

- API - Application Programming Interface
- ADA - Americans with Disabilities Act
- WCAG - Web Content Accessibility Guidelines

- FAQ - Frequently Asked Questions
- AirLabs - Website that provides open access of air traffic control data for research
- Bootstrap framework - open source front-end framework to build websites
- Google OAuth - Protocol that allows applications to access a user's google data without needing their password.
- Open-Meteo Weather Forecast API - open source API that displays the weather forecast in your location
- SQLite - Embedded relational database used for local user data storage
- dB - Decibel - unit of sound pressure level
- JWT- JSON Web Token

1.5 References

<https://www.w3.org/TR/WCAG21/> Web Content Accessibility Guidelines(WCAG) 2.1, 2025, World Wide Web Consortium

<https://airlabs.co/terms-of-service> AirLabs API Terms of Service, 07/20/2021, AirLabs

<https://open-meteo.com/en/features> Open-Meteo, 2026, Open-Meteo

<https://www.ada.gov/law-and-regs/ada/> Americans with Disabilities Act of 1990, 1990, U.S. Department of Justice

<https://developers.google.com/maps> Google Maps JavaScript API

<https://developers.google.com/identity> Google Identity Services (Sign-In)

2. Overall Description

2.1 System Analysis

FlightSound is designed to address the challenge faced by individuals who live near airports or beneath major flight paths, specifically, the difficulty of predicting and managing airplane noise. Currently, residents rely on inconsistent sources of information about air traffic patterns, and there is no widely accessible tool that combines real-time air traffic with estimated noise levels and future flight path projections.

Goals of the Project

- Provide users with real-time visualizations of aircraft locations and movements
- Display estimated noise levels based on aircraft type, altitude, and position
- Offer predictions of future flight paths to help users plan quiet periods for rest, work, or leisure
- Present all data in a clear, accessible, and user-friendly format

Technical Hurdles

- Integrating real-time visualizations of aircraft locations and movements
- Accurately estimating noise levels based on flight metadata
- Displaying a large number of aircraft on an interactive map without performance issues
- Ensuring the interface is accessible and usable across a range of devices
- Presenting future flight path predictions using available datasets

Approach to Overcoming Hurdles

- Utilize stable, documented public APIs to retrieve flight and aircraft data
- Apply standardized aviation noise-estimation formulas to generate consistent noise values
- Implement efficient map-rendering techniques to support large data sets
- Follow accessibility guidelines to support a diverse user base
- Use predictive modeling based on available flight metadata to approximate future paths

Current Prototype Progress:

The system currently supports real-time flight visualization, backend and frontend integration, and noise level estimation. However, geographic filtering is still being refined, as some flights outside the intended airport radius may still be displayed. Additional features such as weather integration and predictive flight paths are not yet fully implemented.

2.2 Product Perspective

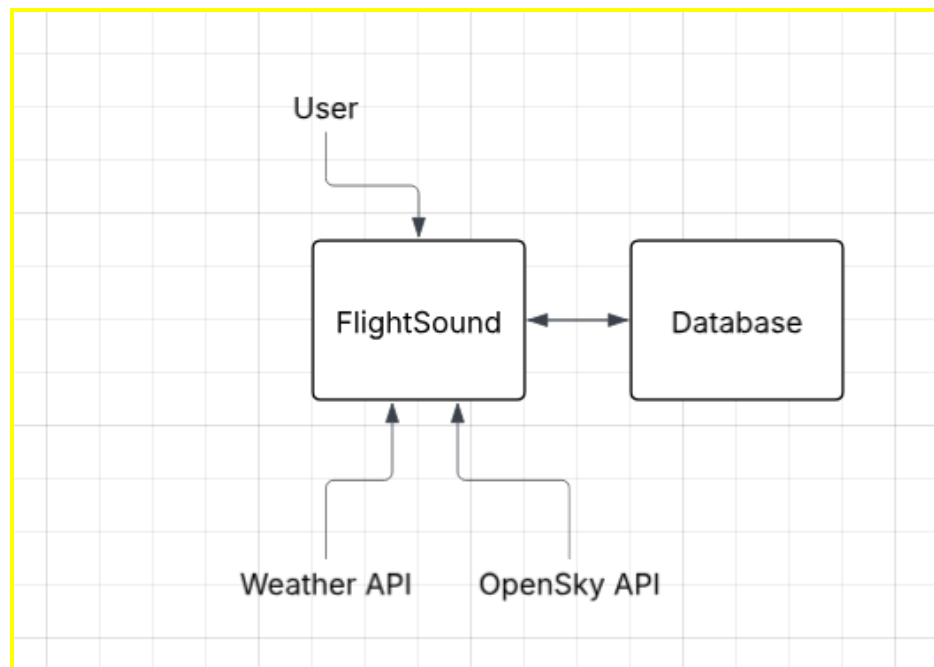
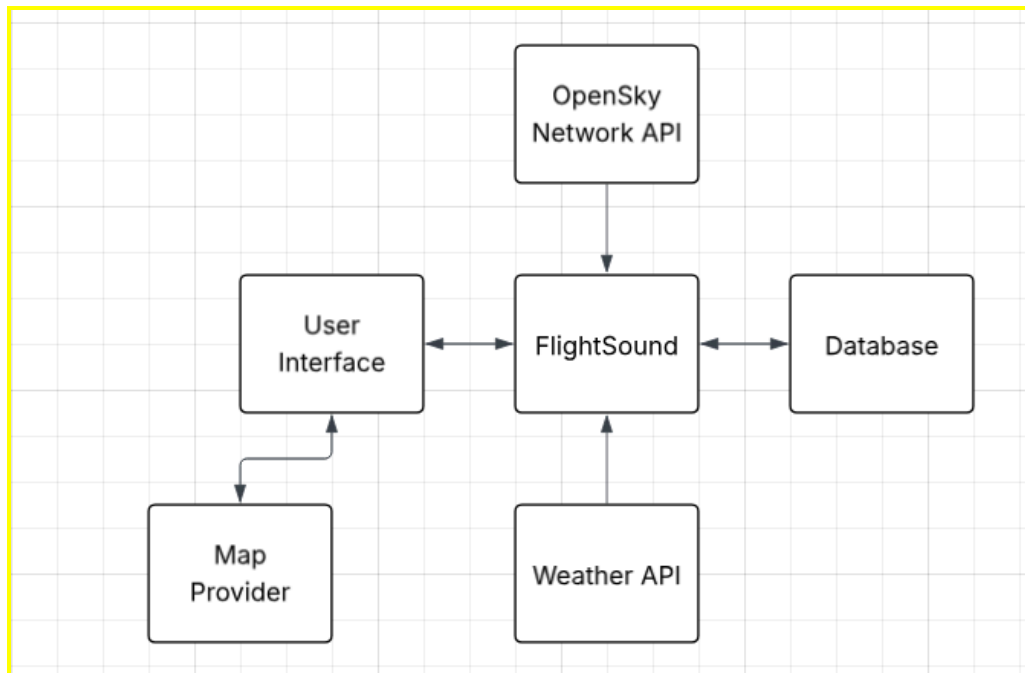
FlightSound is a standalone web-based application that relies on several external services to provide real-time aircraft information, predicted noise levels, and future flight paths. While the system does not depend on any internal organizational software, it interacts with multiple third-party data sources and tools to function properly

External System Relationships

FlightSound communicates with the following systems:

- AirLabs API - Provides real-time aircraft data, including position, altitude, and flight identifiers
- Open Meteo API - Supplies current weather conditions and daily high/low temperatures (free, no key required)
- Google Maps JavaScript API - Renders the interactive map and aircraft markers
- Google Identity Services - Handles user authentication via Google Sign-In
- SQLite - Stores user accounts and preferences locally on the server

FlightSound acts as the central system that gathers information from external APIs, processes it, and delivers the output through a web interface. The system does not modify external data sources; it only consumes them to produce visualizations and noise predictions.

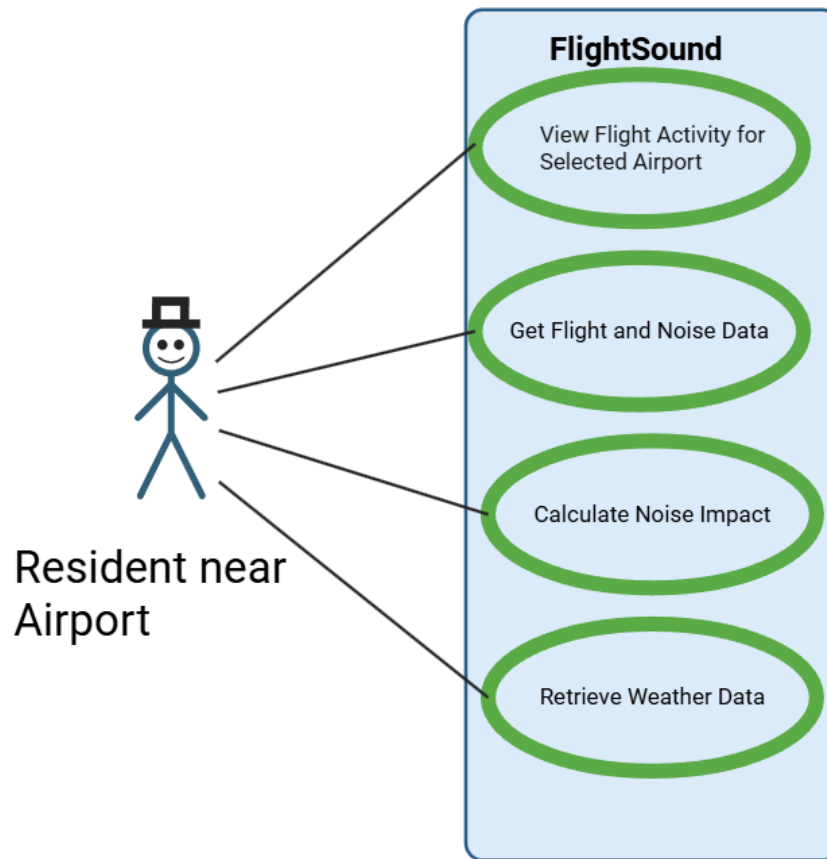


This diagram shows how flight sound fits into its environment: as a central application supported by external data sources and accessed by users through a web interface

Note: The system previously referenced the OpenSky API but now uses the AirLabs API for improved reliability.

2.3 Product Functions

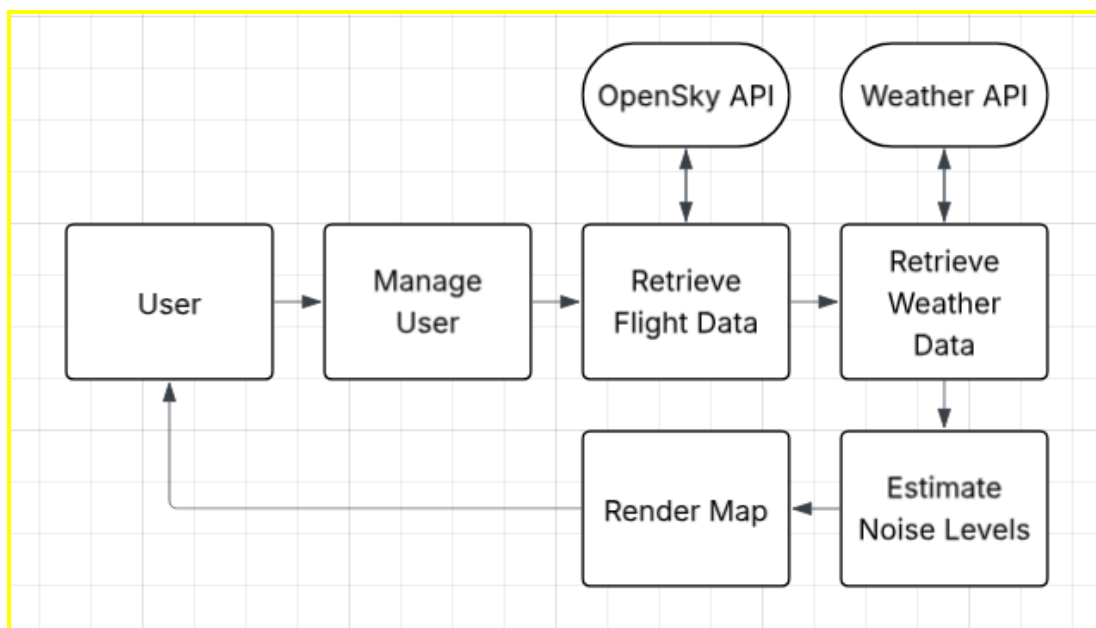
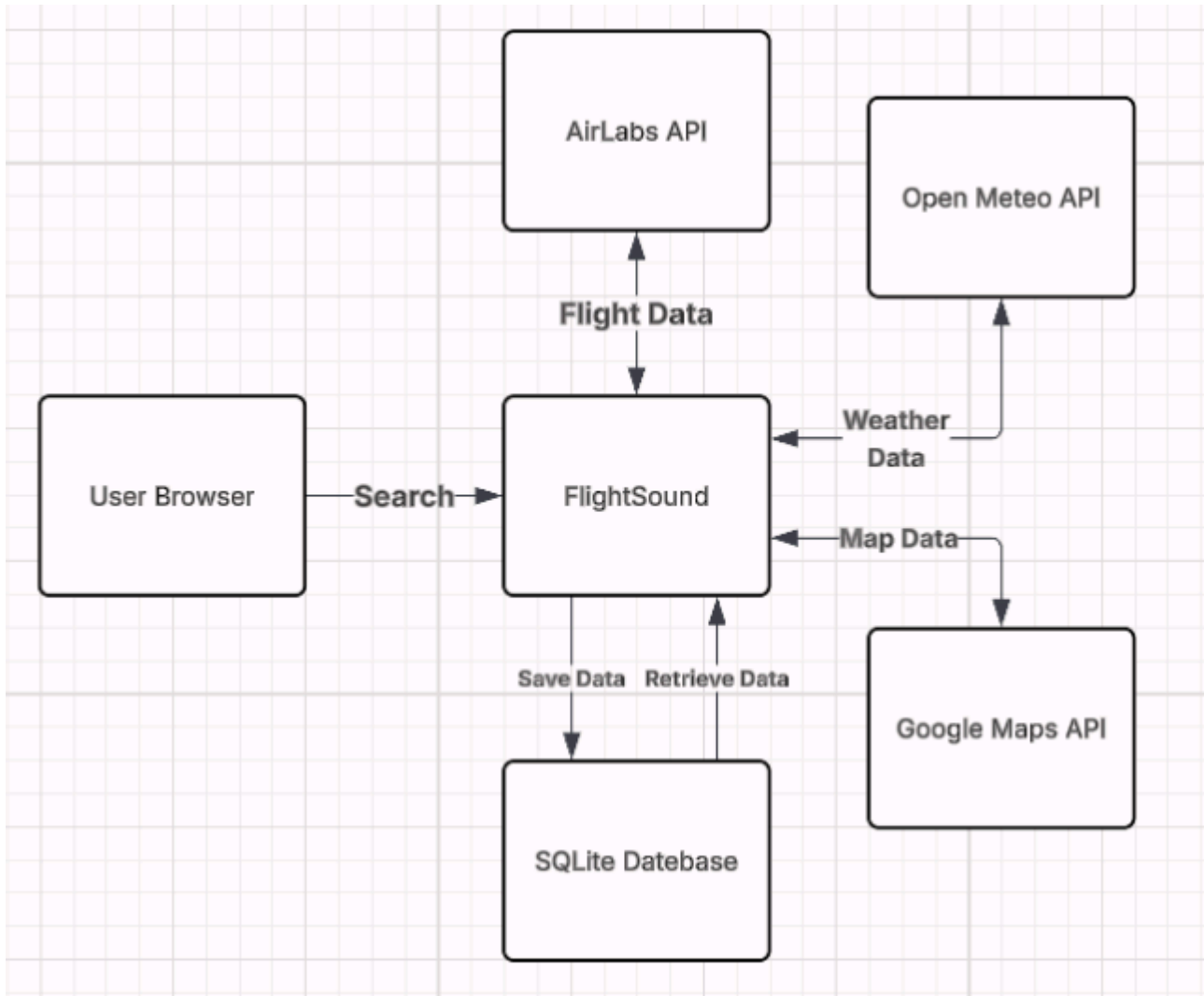
FlightSound provides several core functions that allow users to view aircraft activity and understand the associated noise levels. These functions work together to collect data from external sources, process it, and present it visually to the user



The major functions in the system include

1. User Interaction
 - a. The system allows users to access the FlightSound website and interface to view the map, observe aircraft positions, and access noise level information. The system must respond to user inputs and update the display accordingly
2. Manage User Session
 - a. The system manages user interaction sessions to ensure proper communication between the user and the application. This includes handling user requests, loading relevant data, and maintaining system responsiveness during use
3. Retrieve Flight Data
 - a. FlightSound retrieves real-time flight information from the AirLabs API. This data includes aircraft location, altitude, and identification information. The backend processes this data and sends structured results to the frontend for visualization.
4. Retrieve Weather Data
 - a. The system is planned to retrieve weather information from an external Weather API. This feature is not yet fully implemented in the current prototype.
5. Estimate Noise Levels
 - a. FlightSound calculates estimated aircraft noise levels using available flight data such as altitude. This feature is implemented in the backend and the results are displayed in the user interface.
6. Render Map Visualization

- a. The system displays an interactive Google Map that shows aircraft positions and estimated noise levels. Users can view flights and interact with the map in real time.



2.4 User Classes and Characteristics

FlightSound is designed to support multiple user classes with different levels of experience, technical knowledge, and usage needs. These user classes are defined based on their purpose for using the system and the features they are most likely to use

1. General Users (Primary User Class)

General users represent the largest and most important user group. These users are typically residents living near airports or under common flight paths who want to understand aircraft noise levels and flight activity

Characteristics

- Limited to moderate technical expertise
- Use the system occasionally or regularly, depending on their needs
- Primarily interested in viewing aircraft locations and estimated noise levels
- Require a simple, intuitive, and easy-to-understand interface

Importance: This is the most important user class because FlightSound is primarily designed to help everyday users understand and plan around aircraft noise.

2. Aviation Enthusiasts

Aviation enthusiasts use the system to observe aircraft movements and learn more about flight activity.

Characteristics

- Moderate to high technical knowledge of aviation concepts
- Frequent system usage
- Interested in detailed flight information such as aircraft type, altitude, and flight paths
- Comfortable interacting with advanced system features

Importance: This is a secondary user class that benefits from the system's flight tracking and visualization features

3. Researchers

Researchers use FlightSound for data analysis, environmental studies, or academic purposes related to aircraft noise and flight patterns

Characteristics

- High technical and analytical expertise
- May use the system to observe trends in flight activity and noise level
- Interested in accurate and consistent data presentation

Importance: This is a specialized user class that may rely on the system for professional or academic work

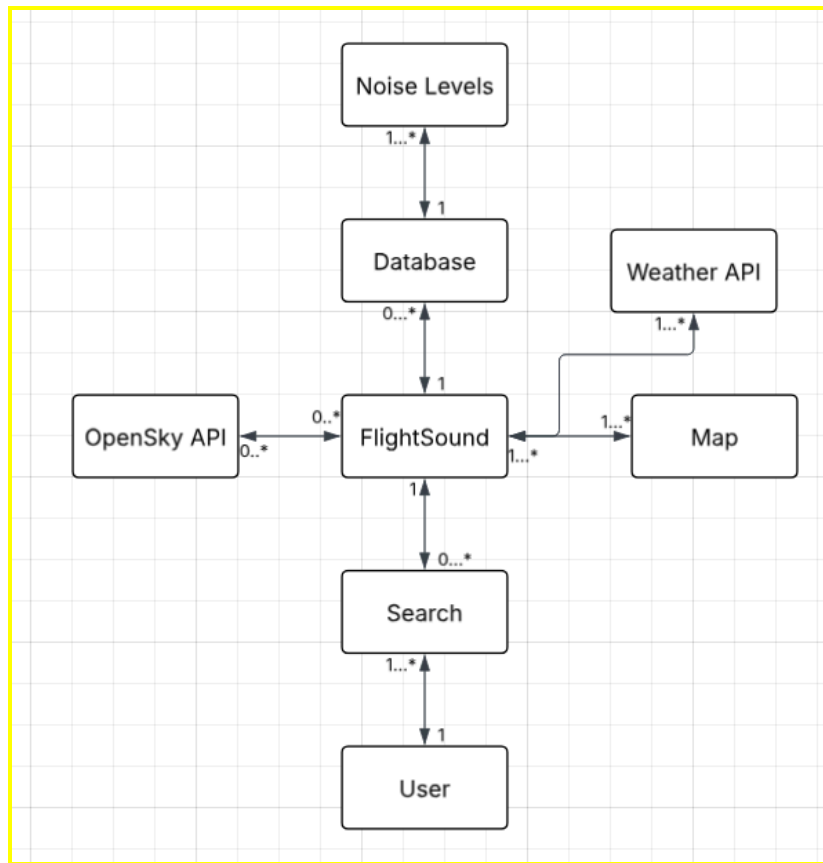
4. System Administrator

The administrator is responsible for maintaining the system and ensuring it operates properly

Characteristics

- High technical expertise
- Responsible for system monitoring, maintenance, and troubleshooting
- May manage system configuration and database integrity

Importance: This is a support user class that ensures the system remains functional but is not the primary end user



2.5 Operating Environment

FlightSound will operate in the following environment:

- Web-based application accessed through modern browsers (Chrome, Safari, Firefox, Edge).
- Hosted on a cloud-based web server.
- Database hosted remotely or on a managed server.
- Usable on:
 - Desktop computers
 - Laptops
 - Tablets
 - Mobile Devices (via browser)

The current version of the system operates as a working prototype where users can search for an airport, view nearby flights on a map, and see estimated noise levels. The frontend and backend are connected and actively exchanging data.

2.6 Design and Implementation Constraints

The development of FlightSound is affected by the following technical, legal, and operational constraints:

Regulatory and Compliance

- Must comply with ADA accessibility requirements (WCAG 2.1 Level AA).
- Must follow the API provider's terms of service.
- Any collected user data must follow basic privacy principles.

Technical and Hardware Constraints

- Must run on common devices.
- Performance should remain stable on lower-powered devices.
- Browser compatibility is required across modern web browsers.
- Real-time map rendering must not freeze or crash.

System Performance and Reliability

- Must support multiple users at the same time.
- Must handle concurrent API requests without errors.
- Should remain available during peak usage times.

Security Requirements

- Must protect against common web vulnerabilities.
- API keys must be stored securely
- Location-based data must not be misused.

Another constraint observed during development is dependency on external APIs for live flight data. The system's functionality is limited if the API is unavailable or returns incomplete data.

2.7 User Documentation

FlightSound will include:

- Interactive map.
 - Includes labels showing flight paths, aircraft details, and noise estimates.

2.8 Assumptions and Dependencies

Assumptions

- Users have reliable internet access for real-time data retrieval
- Modern web browsers are available.
- AirLabs API and Weather APIs remain publicly accessible.
- Aircraft position and altitude data continue to be available.
- Hosting services remain operational.

If any of these assumptions change (e.g., APIs become restricted or unavailable), system requirements and functionality would need to be revised.

Third-party Components

- AirLabs API (flight tracking data)
- Open-Meteo Weather Forecast API (environmental data)
- Google Maps API (map rendering and geolocation)
- Hosting platform (needed to display weather)
- GitHub (version control and project management)

Changes to pricing, access limits, API structure, or service availability may require modifications.

Software Dependencies

- Modern web technologies (JavaScript frameworks, server-side runtime, SQLite database)

- Reusable open-source libraries for map rendering or data visualization

Any major updates, deprecations, or incompatibilities in these components may impact system design and performance requirements.

2.9 Apportioning of Requirements

The initial version of FlightSound will focus on real-time flight visualization and noise estimation.

The following features may be deferred to future versions:

- Mobile application.
- Historical multi-year noise trend analytics.
- Personalized user accounts and saved preferences.
- Push notifications for high-noise periods.
- Machine learning-based noise prediction improvements.
- Community reporting features.
- Filtering & Search Module (filter by altitude/noise/aircraft type, search flight ID)
- Improved geographic filtering for accurate airport-based flight display

3. External Interface Requirements

3.1 User Interfaces

Layout and Navigation

- Top navigation bar with:
 - FlightSound logo
 - Home
 - Settings
 - About
 - Contact links
 - Google Sign-In button
 - user badge (when signed in)
 - dark mode toggle
- Search bar with airport autocomplete (name or IATA code)
- Preferences bar:
 - default airport input
 - location address input
 - coordinate toggle
 - My Location button
 - Save button
- Two-column dashboard:
 - left column (map + controls)
 - right column (flight list sidebar + weather panel)
- Map controls:
 - Noise Impact & Quiet Times button
 - My Location button
 - map type selector (Roadmap/Satellite/Hybrid/Terrain)
- Modals:
 - Settings
 - Info (Best Time Periods)
 - Username selection (first login)

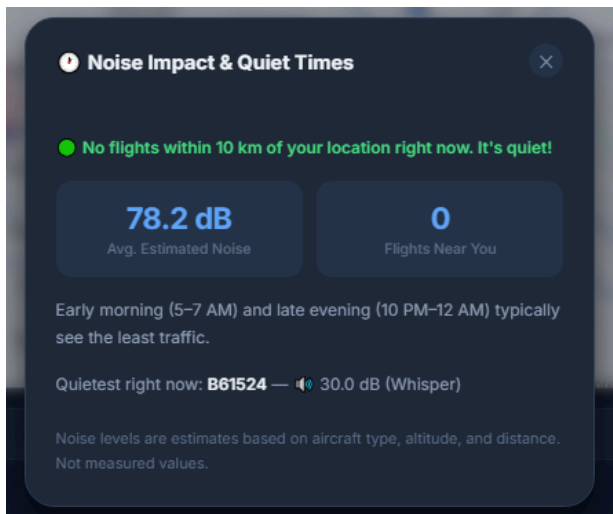
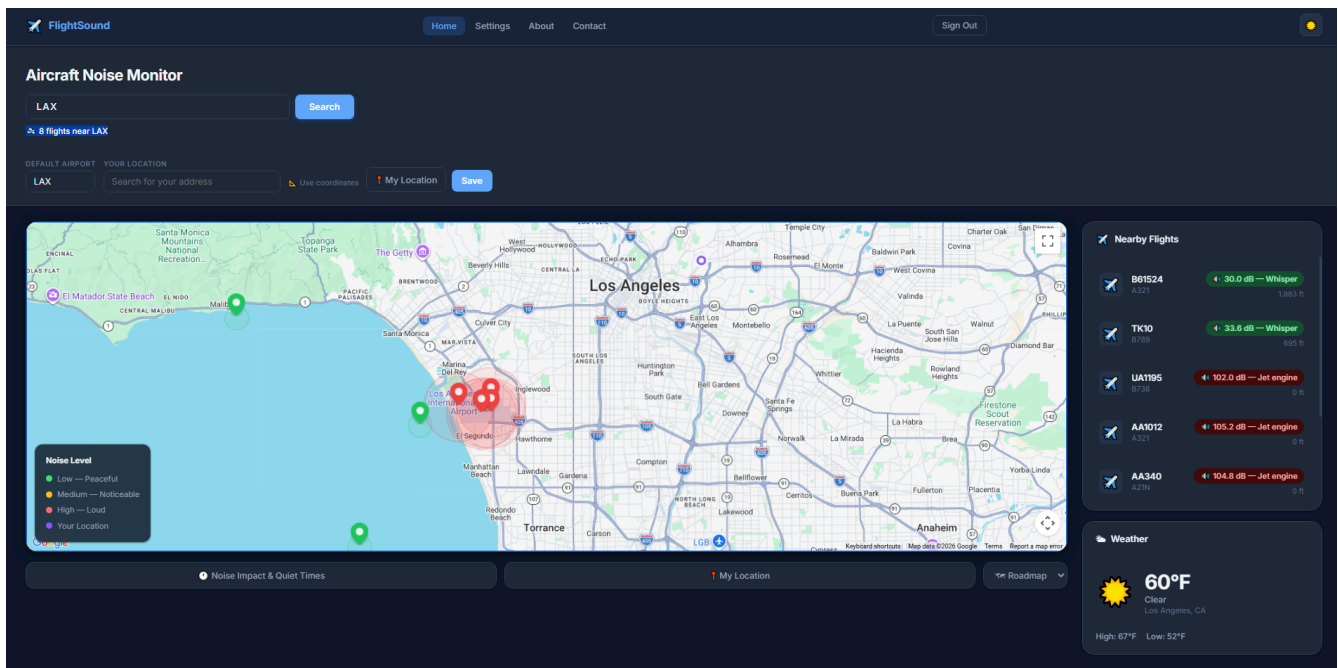
Map Interactions

- Users can zoom and pan the map
- Clicking an aircraft marker opens an info window with flight details
- Noise circles (toggleable) show estimated impact radius per aircraft
- Color coding: green = low (<65 dB), yellow = medium (65–84 dB), red = high (≥85 dB)

Error and Status Display

- Status message displayed below the search bar (e.g., "Fetching flights...", "N flight(s) near AIRPORT")
- Critical errors highlighted in red
- Empty state messages when no flights are found

Note: Navigation buttons (Home, About, Contact), future flight path viewer, and some settings options are currently placeholders without full functionality.



DEFAULT AIRPORT: LAX YOUR LOCATION: Search for your address

LATITUDE: 34.07 LONGITUDE: -118.17

Hide coordinates My Location Save

Accessibility Requirements (ADA Compliance)

The system must conform to the Americans with Disabilities Act (ADA) and accessibility standards, including:

- Support for screen readers
- Proper use of alt text for images
- High contrast color options
- Scalable text (zoom support)

- Keyboard-only navigation support
- Captions for multimedia content
- Clear focus indicators for interactive elements

The system will follow WCAG 2.1 Level AA guidelines where applicable

Note: Some interface components such as navigation buttons (Home, Settings, About, Contact), weather display, and future flight path features are currently placeholders and do not have full functionality implemented in the prototype.

3.2 Hardware Interfaces

Supported device types

- Desktop and laptop computers (Windows, macOS, Linux) via modern browsers.
- Mobile devices and tablets (iOS, Android) via mobile browsers or progressive web app (PWA).

Display / Screen

- The application renders an interactive map and UI in the browser. The software issues standard CSS rendering commands to the browser, which the client hardware displays.

Mouse & Keyboard

- Mouse events (click, hover) and keyboard input (typing, shortcuts, focus navigation) are captured via standard browser event handlers.

Location services (GPS)

- GPS/location via browser Geolocation API (user permission required, read-only)

Orientation

- If used (e.g., map orientation), the app may read device orientation via Device Orientation APIs (permission-controlled).

Speaker / Audio

- For optional audio alerts or voice feedback, the app uses standard Web Audio / HTML5 audio playback to output sound through device speakers.

3.3 Software Interfaces

Interface	Purpose
GitHub	Repository, team will collaborate and share/store code on GitHub
Airlabs (AirLabs Network)	Used to access real-time air traffic data.

Visual Studio (Version 18.3.1, from Microsoft)	Development environment
Kiro (Version 0.11.133, from Amazon Web Services)	Agentic development environment
Git (Version 2.53.0, for Windows)	Will be used to track any changes made to the code
Gson 2.10.1: Reads JSON (JavaScript Object Notation)	serialization/deserialization in backend
Open-Meteo API (api.open-meteo.com/v1/forecast)	Current weather + daily forecast, used in noise calculations
Google Maps JavaScript API	Interactive map rendering (key embedded in index.html)
Nominatim API (https://nominatim.org/)	Forward geocoding to convert text address into coordinates
Google Identity Services	Google Sign-In / OAuth (Client ID in auth.js)
SQLite (via JDBC)	User account and preference storage

3.4 Communications Interfaces

- Frontend-to-backend
 - HTTP REST over localhost (development); HTTPS in production
- Backend-to-AirLabs
 - HTTPS GET requests with API key as query parameter
- Backend-to-Open-Meteo
 - HTTPS GET requests, no authentication
- All API responses are JSON
- Backend includes CORS headers (Access-Control-Allow-Origin: *) on all responses
- Recommended production data transfer rate
 - 50 Mbps or higher
- Production deployments shall use TLS 1.2 or higher

4. Requirements Specification

4.1 Functional Requirements

4.1.1 Flight Activity Map

- 4.1.1.1

The system shall display an interactive Google map centered on the coordinates of the selected airport showing real-time aircraft activity.

- 4.1.1.2

The system shall render aircraft markers on the map representing flights within 50 km of the selected airport.

- 4.1.1.3

The system shall allow users to zoom and pan the map.

- 4.1.1.4

The system shall support map type selection: Roadmap, Satellite, Hybrid, and Terrain.

- 4.1.1.5

The system shall display a noise level legend on the map identifying low, medium, high, and user-location markers.

4.1.2 Airport Selection

- 4.1.2.1

The system shall allow users to search for an airport by IATA code (e.g., "LAX") or by partial airport name.

- 4.1.2.2

The system shall display autocomplete suggestions from a predefined airport list as the user types.

- 4.1.2.3

The system shall update the map and flight data to reflect the selected airport upon search submission.

- 4.1.2.4

The system shall display an error message if the entered airport code is not in the supported airport list.

4.1.3 Geographic Filtering

- 4.1.3.1

The system shall exclude flights whose position coordinates are more than 50 km from the selected airport's coordinates, calculated using the Haversine formula.

- **4.1.3.2**

The system shall exclude flights that have no position data (latitude and longitude both equal to 0.0).

4.1.4 Aircraft Marker Display

- **4.1.4.1**

The system shall display one marker per active flight within the 50 km radius.

- **4.1.4.2**

The system shall color each marker based on estimated noise level: green for low (<65 dB), yellow for medium (65–84 dB), red for high (≥85 dB).

- **4.1.4.3**

The system shall update marker positions when new flight data is fetched.

- **4.1.4.4**

The system shall display a noise impact circle around each marker when the "Show Noise Circles" setting is enabled.

4.1.5 Aircraft Details

- **4.1.5.1**

The system shall display an info window when a user clicks an aircraft marker.

- **4.1.5.2**

The info window shall include: flight number, airline name, aircraft type (ICAO code), altitude in feet, estimated noise level in dB, and noise impact classification (low/medium/high).

- **4.1.5.3**

The sidebar shall list all nearby flights with flight number, noise level in dB, and altitude.

4.1.6 Noise Calculations

- **4.1.6.1**

The system shall estimate aircraft noise level in decibels using the formula: $\text{estimatedDb} = \text{baseNoise} - 20 * \log_{10}(\text{distanceFt} / 1000)$, where distanceFt is the 3D slant distance from the user to the aircraft.

- **4.1.6.2**

When user coordinates are provided, the system shall compute slant distance as $\sqrt{\text{horizontalFt}^2 + \text{altitudeFt}^2}$, where horizontal distance is derived from the Haversine formula.

- **4.1.6.3**

When user coordinates are not provided, the system shall use altitude alone as the distance.

- **4.1.6.4**

The system shall use per-aircraft-type base noise values from a lookup table of ICAO codes (e.g., B738 = 98 dB, A320 = 97 dB). Aircraft types not in the table shall default to 85 dB.

- **4.1.6.5**

The system shall enforce a minimum altitude of 200 ft to prevent unrealistic noise values for ground-level flights.

- **4.1.6.6**

The system shall clamp all estimated noise values to the range [30, 130] dB.

- **4.1.6.7**

The system shall classify noise into impact zones: high (≥ 85 dB), medium (65–84 dB), low (< 65 dB).

4.1.7 Weather Data

- **4.1.7.1**

The system shall retrieve current weather conditions for the selected airport from the Open-Meteo API using the airport's latitude and longitude.

- **4.1.7.2**

The system shall display the following weather data in the weather panel: location name, current temperature in Fahrenheit, weather condition (e.g., Clear, Rain, Foggy), and daily high/low temperatures.

- **4.1.7.3**

The system shall support weather retrieval for the 47 airports defined in the WeatherService coordinate table.

- **4.1.7.4**

If the weather request fails, the system shall return flight data with a null weather field and log the error server-side; it shall not return an error to the user for the flight data.

4.1.8 User Authentication

- **4.1.8.1**

The system shall support user sign-in via Google Identity Services.

- **4.1.8.2**

Upon successful Google Sign-In, the system shall send the user's Google ID and email to POST /api/auth/google/callback.

- **4.1.8.3**

The system shall create a new user record if the Google ID does not exist, or return the existing record if it does.

- **4.1.8.4**

On first sign-in, the system shall prompt the user to choose a unique username (1–30 characters).

- **4.1.8.5**

The system shall display the username in the navigation bar when the user is signed in.

- **4.1.8.6**

The system shall support sign-out, clearing the local session.

4.1.9 User Preferences

- **4.1.9.1**

The system shall allow authenticated users to save a default airport (IATA code, max 4 characters).

- **4.1.9.2**

The system shall allow authenticated users to save a home location as latitude/longitude coordinates.

- **4.1.9.3**

The system shall allow users to enter a home address, which is geocoded to coordinates via the Google Maps Geocoding API.

- **4.1.9.4**

The system shall allow users to manually enter latitude and longitude coordinates.

- **4.1.9.5**

The system shall persist preferences to the server via POST /api/user/preferences and retrieve them via GET /api/user/preferences?google_id=....

- **4.1.9.6**

The system shall use saved user coordinates for distance-based noise calculation when available.

- **4.1.9.7**

The system shall allow users to filter flights based on minimum noise levels for a flight to be displayed.

- **4.1.9.8**

The system shall allow users to sort the order in which they view flights based on the following factors:

1) Default (received from airlabs) 2) Noise levels 3) Distance from user 4) Altitude 5) Name

4.1.10 Data Storage

- **4.1.10.1**

The system shall store user records in a SQLite database (users.db) with the following fields: google_id (TEXT, unique), email (TEXT, unique), username (TEXT, unique, nullable), preferred_airport (TEXT, nullable), user_lat (REAL, nullable), user_lng (REAL, nullable).

- **4.1.10.2**

The system shall create the users table on startup if it does not exist.

- **4.1.10.3**

The system shall perform safe schema migrations on startup to add missing columns to existing databases without data loss.

4.2 External Interface Requirements

AirLabs API

- **Name:** AirLabs Flights Endpoint
- **Purpose:** Retrieve real-time flight data for a given departure airport
- **URL:** https://airlabs.co/api/v9/flights?api_key={key}&dep_iata={IATA}
- **Method:** GET
- **Input:** IATA airport code (3–4 uppercase letters), API key from environment variable
- **Output:** JSON array of flight objects including: flight_iata, flight_number, airline_iata, aircraft_icao, alt, lat, lng, dep_iata, arr_iata
- **Timeout:** 10 seconds
- **Error handling:** Non-200 responses throw a RuntimeException; missing API key throws IllegalStateException

Open-Meteo API

- **Name:** Open-Meteo Forecast Endpoint
- **Purpose:** Retrieve current weather and daily forecast for an airport location, also used in noise calculations
- **URL:**
https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lng}¤t=temperature_2m,weather_code&daily=weather_code,temperature_2m_max,temperature_2m_min&temperature_unit=fahrenheit&forecast_days=1&timezone=auto
- **Method:** GET
- **Input:** Latitude and longitude from the WeatherService coordinate table
- **Output:** JSON with current.temperature_2m, current.weather_code, daily.temperature_2m_max, daily.temperature_2m_min
- **Timeout:** 10 seconds
- **Error handling:** Failures are logged server-side; weather field in response is set to null

Backend REST API (Internal)

Endpoint	Method	Description	Response
/api/noise?airport={IATA}&user_lat={lat}&user_lng={lng}]	GET	Returns filtered flights with noise data and weather	{ data: [...], weather: {...} }
/api/auth/google/callback	POST	Create or retrieve user by Google ID	User Object (201 created, 200 existing)
/api/users	GET	List all users	Array of user objects
/api/users/{google_id}	GET	Get user by Google ID	User object
/api/users/{google_id}/username	PATCH	Update username	Updated user object
/api/user/preferences	GET	Get user preferences	{ preferred_airport, user_lat, user_lng }
/api/user/preferences	POST	Save user preferences	{ status: "success" }

All endpoints return JSON. All endpoints include CORS headers. Error responses use { "error": "message" } format.

4.3 Logical Database Requirements

Entity: User

Column	Type	Constraints	Description
google_id	TEXT	NOT NULL, UNIQUE	Google OAuth subject identifier
email	TEXT	NOT NULL, UNIQUE	User's Google account email
Username	TEXT	UNIQUE, nullable	User-chosen display name (1-30 chars)
preferred_airport	TEXT	nullable	Default airport IATA code
user_lat	REAL	nullable	User's home latitude
user_lng	REAL	nullable	User's home longitude

- The users table is the only persistent entity in the current system.
- Flight data is not persisted; it is fetched live on each request.
- Weather data is not persisted; it is fetched live on each request.

- The database is accessed by a single server process; no concurrent write conflicts are expected in the current architecture.
- Data retention: user records persist indefinitely until manually deleted.

4.4 Design Constraints

- **Language:** Backend must be implemented in Java. Frontend must be implemented in plain HTML, CSS, and JavaScript (no frontend framework).
- **Build:** No build tool (Maven/Gradle) is used. Compilation is performed with javac using explicit classpath entries for gson-2.10.1.jar and sqlite-jdbc-3.36.0.3.jar.
- **API Key Security:** The AirLabs API key must be provided via the FLIGHTLABS_API_KEY environment variable. It must not appear in any source file or version-controlled file.
- **Airport Support:** The system is constrained to airports defined in the NoiseHandler.AIRPORT_COORDS and WeatherService.COORDS lookup tables. Adding a new airport requires a code change in both classes.
- **Port Assignments:** The backend server runs on port 8080. The frontend development server runs on port 5500. These are not configurable at runtime without code changes.
- **Database Location:** The SQLite database file (users.db) is created in the working directory from which the server is launched.
- **CORS:** *The backend allows all origins () in development. Production deployments should restrict this to the frontend's domain.*

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **5.1.1**

The system shall return a response from GET /api/noise within 5 seconds under normal network conditions (excluding AirLabs API latency).

- **5.1.2**

The frontend shall render up to 100 aircraft markers on the map without visible frame rate degradation on a device with a modern browser and 4 GB RAM.

- **5.1.3**

The backend shall handle at least 10 concurrent HTTP requests without returning errors.

- **5.1.4**

The AirLabs API request shall time out after 10 seconds; the system shall return a 500 error to the client if the timeout is exceeded.

- **5.1.5**

The Open-Meteo weather request shall time out after 10 seconds; the system shall return flight data with a null weather field if the timeout is exceeded.

5.2 Safety Requirements

- **5.2.1**

The system shall not use noise level estimates for any safety-critical aviation decision-making. All noise values are estimates for informational purposes only.

- **5.2.2**

The system shall display a disclaimer that noise levels are estimated and not certified measurements.

- **5.2.3**

The system shall not store or transmit user location data to any third party other than the backend server.

5.3 Security Requirements

- **5.3.1**

The AirLabs API key shall be stored exclusively as an environment variable (FLIGHTLABS_API_KEY) and shall never appear in source code, configuration files, or version control history.

- **5.3.2**

The Google Maps API key is embedded in index.html and is restricted to the application's domain via Google Cloud Console key restrictions.

- **5.3.3**

The backend shall use parameterized SQL queries (PreparedStatement) for all database operations to prevent SQL injection.

- **5.3.4**

User authentication shall rely entirely on Google Identity Services token verification; the backend shall not implement its own password storage or session tokens.

- **5.3.5**

The backend shall validate all required input fields and return 400 errors for missing or blank required parameters.

- **5.3.6**

Production deployments shall use TLS 1.2 or higher for all client-server communication.

5.4 Software Quality Attributes

- **Reliability:**

The system shall degrade gracefully when external APIs are unavailable. A weather API failure shall not prevent flight data from being returned. An AirLabs API failure shall return a 500 error with a descriptive message.

- **Maintainability:**

Adding support for a new airport requires updating two lookup tables (NoiseHandler.AIRPORT_COORDS and WeatherService.COORDS). Adding a new aircraft type requires updating NoiseCalculator.BASE_NOISE_TABLE. No other code changes are required for these extensions.

- **Portability:**

The backend runs on any platform with Java 17+ and the required JARs. The frontend runs in any modern browser without installation.

- **Usability:**

A new user shall be able to search for an airport and view flight noise data within 3 interactions (load page → type airport → click Search).

- **Testability:**

The NoiseCalculator, FlightService, and FlightParser classes are designed with constructor injection to support unit testing with mock dependencies.

5.5 Business Rules

- **5.5.1**

Any user (authenticated or not) may search for airports and view flight noise data.

- **5.5.2**

Only authenticated users (signed in via Google) may save preferences (default airport, home location).

- **5.5.3**

Each Google account may have at most one FlightSound user record, identified by Google ID.

- **5.5.4**

Username must be unique across all users. A username that is already taken shall be rejected with a 409 Conflict response.

- **5.5.5**

The system shall not charge users for access. All data sources used (Open-Meteo, Google Maps free tier) must remain within their free usage limits.

6. Legal and Ethical Considerations

API Terms of Service

- The system uses the AirLabs API under a commercial license. Usage must comply with AirLabs' terms of service, including rate limits and restrictions on redistribution of raw flight data. The system uses the Google Maps JavaScript API and Google Identity Services under Google's Terms of Service. The system uses Open-Meteo under its free, open-data license.

Privacy

- The system collects minimal user data: Google ID, email address, and optional home coordinates. This data is stored in a local SQLite database and is used to support user preferences and personalized noise calculations. No data is sold or shared with third parties. User location data (coordinates) is used solely to improve noise distance calculations and is stored only when the user explicitly saves preferences. Users may use the system without creating an account.

Noise Data Accuracy

- Noise level estimates are computed from a simplified physics model and are not certified acoustic measurements. The system shall not be used to make regulatory, legal, or health-related decisions about noise exposure. Displaying estimated values without this context could mislead users; the system should include appropriate disclaimers.

Accessibility

- The system targets WCAG 2.1 Level AA compliance to ensure usability for people with disabilities, consistent with ADA requirements.

In the current prototype, all user data is stored locally within the system database and is not transmitted to any third-party services beyond what is required for authentication (Google Identity Services) and external API requests (AirLabs and Open-Meteo). No additional data sharing mechanisms have been implemented.

Appendix A: Glossary

API	Application Programming Interface — allows different software systems to communicate with each other.
AirLabs API	A third-party service used to retrieve real-time flight data such as aircraft position, altitude, and speed.
Open-Meteo API	An external API used to retrieve weather data for selected airport locations.
Google Maps API	A service used to display interactive maps and visualize aircraft positions.
Google OAuth	Authentication system that allows users to sign in using their Google account.
Frontend	The part of the system the user interacts with (map, UI, controls).
Backend	The server-side component that processes requests, retrieves data from APIs, and performs calculations.
SQLite	A lightweight relational database used to store user data such as preferences and account information.
Decibel (dB)	A unit used to measure sound intensity.
IATA Code	A three-letter airport identifier (e.g., LAX).
Flight Marker	A visual representation of an aircraft displayed on the map.
Noise Impact	The estimated effect of aircraft noise in a specific area based on calculated values.
REST API	A communication method used between the frontend and backend using HTTP requests.
JSON	A data format used to send and receive information between systems.
WCAG	Web Content Accessibility Guidelines — standards used to make web applications accessible.
ADA	Americans with Disabilities Act — legal requirements for accessibility in software.

Appendix B: Personas

Persona 1: General User (Primary)

Name: Sarah Lee

Age: 20

Occupation: University Student

Tech-Savviness: Low to Moderate (comfortable with web apps and browsers, not deeply technical)

Background: Sarah lives in a suburban neighborhood about two miles from a regional airport. The noise from the airplanes flying over her house disrupt her focus during work calls and make it hard to sleep in on weekends. She uses FlightSound to figure out when the busiest flight windows are so she can schedule her meetings and rest time accordingly.

Goals:

- Quickly see which aircraft are currently flying over or near her neighborhood.
- Understand how loud a given flight is likely to be.
- Save preferences so for the next time she uses Flightsound

Challenges:

- Does not want to interpret raw aviation data (needs noise levels she can understand).
- Will get frustrated when the map is slow to load or aircraft positions are stale.
- Wants the interface to work well on her laptop without requiring any installation.

Persona 2: Aviation Enthusiast

Name: Marcus Whiteman

Age: 29

Occupation: Youtuber (11.2k subs)

Tech-Savviness: Moderate to High (regularly uses flight tracking tools and aviation reference sites)

Background: Marcus has been a plane enthusiast since he was in high school. He uses FlightSound both for the noise estimation features and to observe detailed flight data — aircraft types, headings, altitudes, and flight paths. He often cross-references FlightSound data with other aviation tools and appreciates when the system provides accurate, detailed information so he can use that information for his youtube videos about flight tracking.

Goals:

- View detailed aircraft information including type, altitude, speed, and heading.
- Track specific flights across the map and observe their paths over time.
- Toggle map overlays to compare noise heatmaps against flight path data.

Challenges:

- Wants more data density than a casual user (finds overly simplified interfaces limiting).
- Expects aircraft information to be accurate and up to date.
- May use the system for extended sessions and needs the map to work properly.

Persona 3: Researcher

Name: Dr. Anna Cantrell

Age: 51

Occupation: Environmental Acoustics Researcher at a university

Tech-Savviness: High (experienced with data analysis tools)

Background: Dr. Cantrell studies the impact of aviation noise on residential communities. She uses FlightSound to observe noise level patterns correlated with flight activity near urban airports. She values data consistency and accuracy above all else, and she needs the system to present information in a way that supports her work.

Goals:

- Observe noise level estimates for multiple aircraft over a given area.

- Correlate weather conditions with noise level variations.
- Access consistent, reproducible data for use in academic research.

Challenges:

- Needs noise estimation methodology to be transparent and documented (e.g., the FAQ explaining how noise is calculated).
- Concerned about data accuracy — inconsistencies between displayed values and expected values undermine her work.
- May access the system from institutional networks with stricter browser security policies.

Appendix C: User Journey Map for FlightSound

Journey for the two actors:

1. **Resident User:** wants to check the nearby airport flights and noise level.
2. **Signed-In User:** wants to save preferences and get a personalized experience when they return.

Each journey includes **stages, user actions, emotions, touchpoints, and opportunities for improvement.**

Example User Journey Map (Table Format)

Stage	User Action	Touchpoints	User Emotion 😊 😐 😞	Opportunities for Improvement
Resident User: Check nearby Flight and Noise Levels				
Open Website	Navigate to FlightSound	Browser	😊 Curious	Ensure fast load time
View Default Map	Map loads	Interactive Map	😊 Neutral	Show a guide for new users
Search for Airport	Type airport name in search bar; select from dropdown	Search bar, Dropdown menu	😊 Engaged	Auto search for the nearest airport to the user location
View the Flights	Map populates with color coded marker	Map Markers, Flight List	😊 Engaged	Make the markers planes
View the Noise Levels	Click a flight marker or row to open info popup showing dB level, altitude, aircraft type	Map Info, Flight List	😊 Interested	Add a picture of the aircraft
Check Weather	Weather Panel updates with the current conditions for the selected airport	Weather Panel	😊 Neutral	Weather context helps user understand if conditions may affect noise
Check the Quiet Times	Click the "Noise Impact and Quiet Times" button to see noise summary	Noise Impact and Quiet Times Button	😊 Satisfied	Be able to add the quiet times to their calendar
Signed-In User: Save Preferences				
Open Website	Navigate to FlightSound	Browser	😊 Curious	Ensure fast load time
Sign-in with Google	Click Google Sign-in button	Navbar, Google Sign-in Button	😊 Satisfied	Fast sign-in time
Set Default Airport	Enter preferred airport code	Preference Bar, Airport Input	😊 Engaged	Airport code autocomplete
Set Home Location	Click "My Location" button or type address in the location field	Preference Bar, Location Input, Geolocation	😊 Neutral	Address autocomplete
Save Preferences	Click "Save" button; confirmation message appears briefly and the map updates	Save Button, Status Message	😊 Satisfied	Add a schedule feature to set their quiet times
Return Visit	Reopen FlightSound; map auto-centers on saved airport	Map, Flight List, Preference Bar	😊 Happy	Prompt a "Welcome Back" message
Sign-Out	Click "Sign-Out" button	Navbar, Sign-Out Button	😊 Neutral	Prompt a "Signed-Out" message